Technical Analysis of a

Sutton–Barto Reinforcement Learning Gridworld Implementation

Dynamic Programming, Policy Evaluation, Policy Iteration, and Value Iteration

Technical report based strictly on the uploaded Jupyter Notebook

Contents

1 Introduction **2**

2 Project Overview **2**

3 Technical Foundations **3**

4 Data and Input Processing **4**

5 System Architecture and Code Organization **4**

5.1 Dependencies and shared utilities 4

5.2 Execution flow 5

6 Detailed Methodology **5**

6.1 Part 1: Figure 5.3 gridworld 5

6.2 Part 2: Exercise 15.3 6

6.3 Part 3: Policy evaluation 6

6.4 Part 4: Policy iteration 7

6.5 Part 5: Value iteration 7

7 Mathematical Formulation **8**

7.1 Uniform-policy evaluation 8

7.2 Policy evaluation under a fixed deterministic policy 8

7.3 Action-value comparison 8

7.4 Convergence threshold 8

8 Optimization and Learning Procedure **9**

9 Implementation Details **9**

9.1 State representation 9

9.2 Action representation 9

9.3 Update order 9

9.4 Output persistence 9

9.5 Notebook portability 10

10 Execution Workflow **10**

11 Design Decisions and Rationale **10**

12 Computational Considerations **11**

13 Reproducibility and Reliability **11**

14 Limitations and Technical Considerations **11**

15 Overall Technical Assessment **12**

Conclusion **13**

Abstract

This report provides a rigorous software and methodological analysis of a compact Jupyter Notebook implementing several dynamic-programming exercises from reinforcement learning. The notebook defines a common 5×5 gridworld, implements the special transition structure used for the Sutton–Barto Figure 5.3 setting, evaluates a uniform random policy, performs policy iteration, and performs value iteration. The analysis reconstructs the data flow, state and action representations, Bellman equations, convergence tests, update schemes, policy extraction, persistence of outputs, and practical implementation choices. A separate notebook section labelled Exercise 15.3 is explicitly identified as a placeholder rather than a completed exercise; no unsupported completion claim is made. The report therefore distinguishes carefully between implemented functionality, textbook theory, and conclusions that cannot be established from the source alone.

1 Introduction

The uploaded notebook is a small reinforcement-learning software artifact centered on dynamic programming in finite Markov decision processes. Its implementation uses a two-dimensional grid as the state space and a four-action movement model. The code is organized as a sequence of notebook sections corresponding to five assignment components: the gridworld value function associated with Figure 5.3, Exercise 15.3, policy evaluation, policy iteration, and value iteration.

The main value of the notebook lies in how it translates classical reinforcement-learning recursions into executable numerical procedures, rather than in the size of the codebase. The implementation exposes the relationship between a mathematical Bellman equation and a concrete Python loop over states and actions. It also uses a shared transition model for several closely related dynamic-programming algorithms, making their differences easy to compare.

This report describes the implementation as provided. It does not reconstruct omitted experiments, add new assignments, or infer unstated design intentions. Where a rationale is inferred from the implementation, that inference is labelled as such. Where the notebook does not contain enough information, the report states the limitation explicitly.

2 Project Overview

The notebook contains four executable reinforcement-learning components built on a common gridworld representation.

The common state space is the Cartesian product of five row indices and five column indices:

$$\mathcal{S} = \{0, 1, 2, 3, 4\} \times \{0, 1, 2, 3, 4\}, \qquad |\mathcal{S}| = 25. \tag{1}$$

The action space contains four discrete moves, encoded as integers and displayed symbolically as up, right, down, and left. The movement offsets are stored in the `moves` dictionary as integer row and column increments.

The notebook uses two scalar hyperparameters shared by the dynamic-programming proce-

Table 1: Implementation inventory derived from the notebook.

Component	Main implementation	Role
Common utilities	`states`, `actions`, `moves`, `show_policy`, `save_result`	Shared representation, policy display, and CSV persistence
Figure 5.3 gridworld	`part1_transition`, `solve_part1`	Iterative solution of the equiprobable-policy Bellman expectation equation with special teleport states
Exercise 15.3	Placeholder cell	Indicates a framework, but does not implement the exercise-specific dynamics
Policy evaluation	`transition`, `policy_evaluation`	Iterative evaluation of a uniform random policy on the ordinary gridworld
Policy iteration	`policy_iteration`	Alternating policy evaluation and greedy policy improvement
Value iteration	`value_iteration`	Direct Bellman optimality updates followed by greedy policy extraction

dures:

$$\gamma = 0.9, \qquad \theta = 0.001. \tag{2}$$

Here γ is the discount factor and θ is the convergence threshold used in the stopping conditions.

3 Technical Foundations

The code is an implementation of finite Markov decision process (MDP) dynamic programming. An MDP may be represented by a tuple

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma), \tag{3}$$

where $\mathcal{S}$ is the state set, $\mathcal{A}$ is the action set, P is the transition model, R is the reward model, and $\gamma \in [0, 1)$ is the discount factor.

For a state s, action a, next state s', and reward r, the notebook's deterministic transition functions can be described as

$$P(s' \mid s, a) = \mathbf{1}\{s' = f(s, a)\}, \tag{4}$$

with reward $R(s, a)$ returned together with the next state by the corresponding Python function.

For a stochastic policy $\pi(a \mid s)$, the state-value function obeys the Bellman expectation equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) V_\pi(s') \right]. \tag{5}$$

Because the notebook uses deterministic movement for each selected action, the inner

expectation over s' collapses to the single next state returned by the transition function. For the uniform random policy used by the notebook, $\pi(a \mid s) = 1/4$, so the update becomes

$$V(s) = \frac{1}{4} \sum_{a \in \mathcal{A}} \left[R(s,a) + \gamma V\big(f(s,a)\big) \right]. \tag{6}$$

For optimal control, the Bellman optimality operator is

$$V_*(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma V_*\big(f(s,a)\big) \right]. \tag{7}$$

Policy iteration alternates between evaluating a fixed policy and improving it greedily with respect to the current value function. Value iteration instead repeatedly applies the optimality operator and extracts a greedy policy after convergence.

4 Data and Input Processing

There is no external dataset loader in the notebook. The computational input is generated directly from program constants and Python data structures. The state list is constructed by enumerating all row-column pairs. This gives a deterministic 25-state environment without a separate file-based input stage.

The transition model for the ordinary gridworld is implemented by taking the current row and column, applying the action-specific offset, and checking whether the resulting coordinate remains inside the 5×5 boundary. When the move would leave the grid, the next state is kept equal to the current state and the reward is set to -1. Otherwise, the next state is the moved-to cell and the reward is 0.

The Part 1 transition function intentionally differs. Two special states, $(0, 1)$ and $(0, 3)$, produce deterministic jumps independent of the selected movement offset. The first jump targets $(4, 1)$ with reward 10, while the second targets $(2, 3)$ with reward 5. This special handling is a key part of the Figure 5.3 environment and is not reused by the ordinary `transition` function used later in the notebook.

The notebook uses no feature engineering, tokenization, normalization, batching, or learned representation. The state is already a compact symbolic coordinate, and the numerical representation of a value function is a 5×5 NumPy array indexed directly by the state tuple.

5 System Architecture and Code Organization

The notebook has a simple, mostly monolithic structure: reusable definitions appear near the beginning, followed by separate sections for each assignment part.

5.1 Dependencies and shared utilities

The notebook imports NumPy, pandas, Matplotlib, and the Python `time` module. In the uploaded source, NumPy is used for numerical arrays and convergence calculations, while pandas is used for displaying policies and writing CSV files. Matplotlib and `time` are imported

but are not used by the shown executable cells. This is a minor cleanliness issue and does not affect the implemented algorithms.

The helper `show_policy(policy)` converts an integer-valued 5×5 policy array into a symbolic arrow grid. The mapping is performed through the `actions` dictionary, and the resulting character matrix is wrapped in a pandas DataFrame for tabular display. The helper `save_result(name,array)` converts a NumPy array into a pandas DataFrame and writes it as a CSV file without row indices.

5.2 Execution flow

The notebook executes from top to bottom, but its conceptual dependencies are simpler than the cell order suggests. Common constants and movement definitions are created first. Part 1 then defines its own transition function and solver. Later sections define a second transition model shared by policy evaluation, policy iteration, and value iteration. The policy-iteration and value-iteration routines also reuse the shared `moves`, `states`, γ, and θ definitions.

Table 2: Notebook-level traceability between major source cells and report topics.

Cells	Symbols / functions	Reported role
1	`GAMMA`, `THETA`, `states`, `actions`, `moves`	Global model representation and convergence settings
1	`show_policy`	Integer policy to arrow-grid conversion
1	`save_result`	CSV output mechanism
3	`part1_transition`, `solve_part1`	Figure 5.3 Bellman expectation updates
5	Placeholder print statement	Exercise 15.3 is not actually specified or solved
7	`transition`, `policy_evaluation`	Uniform-policy evaluation
9	`policy_iteration`	Iterative policy evaluation plus greedy improvement
11	`value_iteration`	Bellman optimality updates plus policy extraction
12	Final-output markdown	States that CSV files can be downloaded from the Colab session

6 Detailed Methodology

6.1 Part 1: Figure 5.3 gridworld

The function `part1\_transition(state,action)` defines the transition model for Part 1. It first checks whether the current state is one of the two special states. If so, the corresponding teleport transition is returned immediately. Otherwise, the action is translated into a coordinate offset and applied to the current position.

The boundary test is explicit: any row below zero, above four, column below zero, or above four is treated as an attempted move outside the grid. In that case, the state remains

unchanged and the reward is -1. Valid moves receive zero reward.

The solver initializes V to all zeros. At each iteration it first creates `old = V.copy()`, which makes the update synchronous: every state in the current sweep is computed from values belonging to the previous sweep. The code then evaluates all four actions with equal probability. The exact implemented recurrence is

$$V_{k+1}(s) = \frac{1}{4} \sum_{a \in \mathcal{A}} \left[R(s,a) + \gamma V_k(f(s,a)) \right]. \tag{8}$$

The iteration stops when the maximum absolute change between two successive arrays is less than θ:

$$\max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \theta. \tag{9}$$

This provides an iterative numerical solution of the Bellman expectation equation for an equiprobable random policy. The implementation does not explicitly construct a transition matrix or reward matrix; instead, it evaluates the model procedurally through `part1\_transition`.

6.2 Part 2: Exercise 15.3

The relevant notebook cell contains only a comment-labelled placeholder and a print statement reporting that a framework has been completed. No exercise-specific reward function, transition dynamics, mathematical recurrence, solver, or result is implemented in that cell. The comment instructs a future modification of the reward and transition definitions if a particular textbook edition uses a different figure.

Consequently, the notebook contains a placeholder rather than an implementation of Exercise 15.3. A technically accurate report must not reinterpret the print statement as evidence that the exercise itself has been solved.

6.3 Part 3: Policy evaluation

The ordinary `transition` function defines the baseline gridworld without the special teleport states. The policy-evaluation routine then initializes a zero value function and repeatedly updates every state by averaging the one-step return over all four actions. Because the code uses a fixed coefficient 0.25 for every action, the policy being evaluated is the uniform random policy

$$\pi(a \mid s) = \frac{1}{4}, \qquad a \in \mathcal{A}. \tag{10}$$

Unlike Part 1, the function does not copy the value function before each sweep. It stores the old value only for the current state, computes the new value using the current contents of V, and writes the new value back immediately. This is an in-place iterative evaluation scheme, closely related to Gauss–Seidel-style updates. This avoids the need for a separate previous-state array.

The stopping criterion tracks a scalar `delta`:

$$\Delta = \max_{s \in \mathcal{S}} |V_{\text{new}}(s) - V_{\text{old}}(s)|, \tag{11}$$

and terminates when $\Delta < \theta$. The theoretical fixed point is the same Bellman expectation solution, although the numerical path to it depends on the in-place update order.

6.4 Part 4: Policy iteration

Policy iteration begins with two zero-initialized 5×5 arrays. The value array stores $V(s)$, while the policy array stores one integer action for each state. Since the policy is initialized to all zeros, the initial policy selects the action encoded as up everywhere.

The first inner loop performs policy evaluation for the current deterministic policy. For each state, the selected action is retrieved from `policy[s]`, the environment returns a single next state and reward, and the update is

$$V(s) \leftarrow R(s, \pi(s)) + \gamma V\big(f(s, \pi(s))\big). \tag{12}$$

Again, updates occur in place, and iteration continues until the maximum single-state change is below θ.

Once the policy evaluation loop converges, the notebook enters the policy-improvement stage. It computes all four one-step action values

$$q(s, a) = R(s, a) + \gamma V(f(s, a)) \tag{13}$$

and replaces the current action with `np.argmax(values)`. The policy is declared stable only when no state changes its selected action. This implements the standard greedy policy-improvement step for the deterministic finite MDP encoded by `transition`.

A subtle but important implementation detail is tie-breaking. NumPy's `argmax` returns the first maximizing index, so when multiple actions have exactly equal action values, the smallest integer action is selected. The resulting deterministic policy is therefore greedy under a specific, deterministic tie-breaking rule.

6.5 Part 5: Value iteration

Value iteration uses a single value array and repeatedly computes the action values for every state. For each state, it applies the Bellman optimality update

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma V_k(f(s, a)) \right]. \tag{14}$$

As in policy evaluation and policy iteration, the code updates the array in place. After the values have converged under the threshold test, a separate pass recomputes the action values and selects a greedy action with `np.argmax`.

The key distinction from policy iteration is that value iteration does not maintain a separate policy during its iterative phase. The policy is extracted only after value convergence. This reflects the difference between repeatedly evaluating a fixed policy and repeatedly applying the Bellman optimality operator.

7 Mathematical Formulation

The entire executable reinforcement-learning logic can be expressed compactly in terms of the shared transition function f and reward function R.

7.1 Uniform-policy evaluation

For the ordinary gridworld, the uniform random policy gives

$$V_\pi(s) = \frac{1}{4}\sum_{a=0}^{3}\left[R(s,a) + \gamma V_\pi(f(s,a))\right]. \tag{15}$$

The implementation approximates this fixed point iteratively. The returned NumPy array is a tabular representation of the state-value function, with the first array dimension corresponding to the row coordinate and the second corresponding to the column coordinate.

7.2 Policy evaluation under a fixed deterministic policy

During policy iteration, the policy supplies a single action $\pi(s)$ per state, so the Bellman expectation equation simplifies to

$$V_\pi(s) = R(s,\pi(s)) + \gamma V_\pi(f(s,\pi(s))). \tag{16}$$

The code solves this fixed-point equation numerically through repeated in-place sweeps.

7.3 Action-value comparison

The policy-improvement and value-iteration stages both rely on the same one-step look-ahead quantity,

$$q_V(s,a) = R(s,a) + \gamma V(f(s,a)). \tag{17}$$

For policy improvement, the action maximizing $q_V(s,a)$ is selected as the next policy action. For value iteration, the maximum itself becomes the updated state value.

7.4 Convergence threshold

The notebook uses an absolute sup-norm-style stopping condition,

$$\|V_{k+1} - V_k\|_\infty = \max_{s\in\mathcal{S}} |V_{k+1}(s) - V_k(s)| < \theta. \tag{18}$$

This criterion determines when the numerical iteration stops; it does not imply exact equality with the theoretical fixed point. The selected $\theta = 0.001$ controls the precision at which the iterative routines stop.

8 Optimization and Learning Procedure

The notebook contains no gradient-based optimization, neural-network training, stochastic-gradient descent, backpropagation, or learned parameter vector. The word "learning" in this project therefore refers to dynamic-programming estimation of value functions and policies rather than parameterized function approximation.

The implemented procedures are fixed-point algorithms over a finite state space. Their only continuous scalar control parameter is the discount factor, while the main stopping control is θ. The value arrays are initialized to zeros for all implemented solvers. There is no explicit random initialization and no call to a pseudorandom-number generator in the supplied source.

9 Implementation Details

9.1 State representation

The state is represented as a tuple (r, c). Python tuples are used as keys into the transition functions, while NumPy arrays use the tuple directly as a two-dimensional index. This creates a compact correspondence between the mathematical state $s = (r, c)$ and the memory layout of the tabular value function.

9.2 Action representation

The action integer is the primary computational representation. A second dictionary maps the same action integers to arrow characters for human-readable policy display. A third dictionary maps the integers to row-column increments. This keeps policy storage numeric while allowing the displayed policy to use intuitive directional symbols.

9.3 Update order

The notebook contains two distinct update styles. Part 1 uses a copied previous value array, producing a synchronous sweep. The later algorithms modify V in place, so values computed earlier in a sweep can influence values computed later in the same sweep. This difference does not change the underlying Bellman fixed-point definition, but it does affect the numerical iteration path and potentially the number of sweeps required for termination.

9.4 Output persistence

The output helper writes NumPy arrays to CSV files through pandas. Part 1 saves the gridworld value table, policy evaluation saves its value table, and policy iteration and value iteration each save both the value table and the corresponding policy table. No figures are generated or saved by the provided source, despite the unused Matplotlib import.

9.5 Notebook portability

The code is compatible with a standard Python 3 notebook environment and depends only on commonly available numerical/data libraries for the shown functionality. The final Markdown cell states that generated CSV files can be downloaded from the Colab session. This indicates an intended Google Colab usage context, although the algorithms themselves are not tied to Colab-specific APIs.

10 Execution Workflow

The execution proceeds as follows. The notebook first creates the environment-independent constants and utility definitions. It then constructs the special Figure 5.3 gridworld transition model and iteratively solves its Bellman expectation equation. The resulting value array is printed and written to CSV.

The Exercise 15.3 section does not alter the environment or execute a solver; it only reports that a framework exists. Execution then proceeds to the standard gridworld transition function. The uniform-policy evaluation routine repeatedly averages the one-step returns of the four actions. Policy iteration then starts with a deterministic policy, alternates fixed-policy evaluation with greedy improvement, and stops when no action changes. Finally, value iteration repeatedly applies the maximization form of the Bellman equation and then performs a separate greedy policy-extraction pass.

The final outputs of the implemented sections are tabular CSV files. Overall, the notebook follows a simple workflow: define the environment, solve the dynamic-programming recurrence, display selected arrays, and save the numerical tables.

11 Design Decisions and Rationale

Several design choices are explicit in the implementation. The use of a 5×5 NumPy array is a natural tabular representation for a finite gridworld, because each state maps to exactly one array cell. It avoids a dictionary-based value store and makes the numerical computations compact and easy to inspect.

The transition functions are procedural rather than represented as large probability tensors. For a deterministic environment with only 25 states and four actions, direct calculation is simple and avoids the memory and indexing overhead of explicitly materializing $P(s' \mid s, a)$.

The shared constants γ and θ are defined once and reused across the algorithms. This reduces accidental inconsistency among the implementations. The choice to use `np.argmax` for action selection also ensures deterministic behavior for ties, which is useful for reproducible policy tables.

The notebook is deliberately compact, but this compactness also limits configurability. The policy-evaluation routine, for example, has no policy argument and is hard-coded to the uniform random policy. Similarly, the transition model is hard-coded for the specific grid size and movement rules rather than being encapsulated in a reusable environment class.

12 Computational Considerations

Let $S = |\mathcal{S}| = 25$ and $A = |\mathcal{A}| = 4$. A full Bellman sweep that evaluates every action for every state has computational cost

$$O(SA), \tag{19}$$

which is constant and very small for this particular environment.

The uniform-policy evaluation routine performs four action evaluations per state and therefore has $O(SA)$ work per sweep. Policy improvement likewise performs $O(SA)$ work per improvement sweep. The policy-evaluation phase inside policy iteration evaluates only one action per state and therefore has $O(S)$ work per sweep. Value iteration has $O(SA)$ work per sweep and a final $O(SA)$ policy-extraction pass.

Because the state space is tabular, the storage requirement for a single value function is $O(S)$, and a policy table adds another $O(S)$ structure. The implementation's actual NumPy arrays are tiny, so memory pressure is negligible in this specific project. These asymptotic statements describe the algorithmic form and do not imply a particular runtime for larger problems.

A practical numerical consideration is the use of in-place updates. They reduce auxiliary storage and can accelerate convergence in some dynamic-programming settings, but they make the result of an intermediate sweep depend on state traversal order. The code traverses `states` in row-major creation order because the list was generated by nested loops over row and column indices.

13 Reproducibility and Reliability

The notebook has several useful reproducibility properties. The state space, action mapping, movement model, discount factor, and convergence threshold are explicit in source code. There is no reliance on an uncontrolled random generator for the implemented algorithms. Array initialization is deterministic, and action tie-breaking through `np.argmax` is deterministic.

At the same time, the uploaded notebook contains no dependency-version lockfile, environment specification, automated tests, or explicit assertion-based verification. Notebook cells also have no stored execution counts or outputs in the supplied file, so the source artifact itself does not provide an execution transcript. Reproducibility is therefore reasonably strong at the algorithm-definition level, but less formal at the software-packaging and verification levels.

Another reliability consideration is the lack of an explicit policy argument in the policy-evaluation function. The function name suggests generic policy evaluation, but the actual body evaluates one fixed uniform policy. That naming mismatch could mislead a future maintainer unless the behavior is documented.

14 Limitations and Technical Considerations

The main limitation is the incomplete Exercise 15.3 section. The notebook does not contain the exercise-specific implementation, so a claim that all assignment parts are fully solved

would not be supported by the source.

A second limitation is that the implementation is specialized to a fixed 5×5 grid and four-direction action set. Generalization to arbitrary environments would require parameterizing the dimensions, action space, transition model, reward model, and policy interface rather than relying on module-level constants and hard-coded boundary values.

A third limitation is that policy evaluation is not a general-purpose routine. It does not accept a policy as an argument, and therefore its scope is narrower than its generic-sounding name implies. This is not an error for the present educational exercise if the intent is specifically to evaluate the equiprobable random policy, but it reduces software reusability.

The source also imports Matplotlib and `time` without using them in the executed cells. Removing unused imports would make the notebook cleaner and reduce ambiguity about its actual dependencies. No evidence in the source suggests that these imports cause a functional problem.

Finally, the notebook does not contain automated tests comparing policy iteration and value iteration against one another or validating Bellman residuals. Such checks could improve software assurance, but their absence should be understood as a reliability limitation rather than evidence that the algorithms are incorrect.

15 Overall Technical Assessment

The implemented core is technically coherent for a small tabular reinforcement-learning exercise. The environment is represented clearly, the special Figure 5.3 transition structure is encoded explicitly, and the three central dynamic-programming ideas—policy evaluation, policy iteration, and value iteration—are represented by recognizably standard Bellman recursions.

The code also demonstrates an important software-engineering pattern: a small shared representation supports multiple algorithms without duplicating the entire environment model. The transition function, state enumeration, action encoding, discount factor, and convergence threshold form a reusable base for the later routines.

The main weakness is not the mathematical core, but the level of completion and generality. Exercise 15.3 is a placeholder, and the later functions are narrowly tailored to the exact gridworld. In addition, the code has limited automated verification and packaging metadata. Within the scope of a compact educational notebook, however, the implemented portions provide a direct and understandable correspondence between reinforcement-learning theory and executable dynamic programming.

Conclusion

The uploaded notebook is a compact tabular reinforcement-learning implementation built around Bellman dynamic programming on a 5×5 gridworld. Its most important computational components are the special Figure 5.3 gridworld solver, uniform-policy evaluation, policy iteration, and value iteration. The notebook expresses these methods directly through state and action loops, deterministic transition functions, NumPy value tables, convergence thresholds, and greedy action selection.

The analysis also establishes a clear boundary between implemented and unimplemented material. The Exercise 15.3 cell is a placeholder and should not be represented as a completed solution. Beyond that limitation, the implemented algorithms are structurally consistent with their corresponding Bellman equations and use deterministic, tabular computations that are easy to trace from theory to code.

Overall, the project provides a clear demonstration of reinforcement-learning fundamentals and of translating mathematical dynamic-programming definitions into executable Python. Its main strengths are conceptual transparency and a direct mapping from algorithms to code. The main opportunities for further engineering maturity are stronger abstraction, explicit policy parameterization, automated verification, and completion of the omitted assignment component.